

L'objectif de ce document est de proposer un mini-TP par semaine pendant les vacances d'été, afin de continuer à pratiquer, et de ne pas arriver à la rentrée en ayant tout oublié de la programmation C et OCaml.

Chaque mini-TP est censé prendre moins d'une heure à finir (et si ça vous prend plus d'une heure, c'est que vous manquez de pratique, donc autant s'y mettre dès maintenant).

Pour chaque mini-TP, il est demandé de repartir de zéro, c'est-à-dire de recréer un nouveau fichier et de tout faire dans ce nouveau fichier, sans copier puis modifier un ancien fichier de code.

1 Installation

Lorsqu'on fait un TP, il est IMPÉRATIF de régulièrement compiler et exécuter son code (il est aussi possible d'interpréter en OCaml). Cela nécessite d'avoir un compilateur C et un compilateur OCaml d'installés.

En cas de souci technique, vous pouvez me contacter par mail pour qu'on essaie de voir ensemble.

1.1 Windows

La manière la plus simple de compiler sous Windows est... d'installer Linux. Inutile toutefois de refaire toute l'installation de votre ordinateur : on peut installer WSL qui permet simplement d'émuler un système Linux. Voir <https://learn.microsoft.com/fr-fr/windows/wsl/install> pour l'installation. Je recommande un système Ubuntu (qui est celui par défaut). Retenez le mot de passe administrateur (*root*) que vous devriez configurer, c'est lui qui permettra d'effectuer de nouvelles installations.

Une fois que c'est installé, on peut taper `wsl` dans un terminal Powershell (ou dans toute autre console Windows, comme celle fournie par VS-Code, par exemple) pour obtenir une console Linux (ce qui suffit largement pour compiler).

Pour la suite de l'installation, voir la partie Linux.

1.2 Linux

Je suppose ici que le système est basé sur Debian (c'est par exemple le cas pour Ubuntu). Si vous utilisez un autre système, je pars du principe que vous êtes assez autonome pour faire les bonnes installations.

Il est généralement recommandé de mettre à jour son système avant de procéder à de nouvelles installations.

```
sudo apt update && sudo apt upgrade
```

Ensuite, pour installer `gcc`, on peut taper :

```
sudo apt install build-essential
```

Pour installer un compilateur OCaml, on peut taper :

```
sudo apt install opam
```

1.3 Mac

Je m'y connais moins en Mac, mais c'est un système basé sur Unix, donc ça reste plus simple de faire les installations que sous Windows.

Il est recommandé d'installer un gestionnaire de paquets (comme Homebrew : <https://brew.sh/>), puis d'installer, comme pour Linux, les paquets nécessaires :

```
brew install gcc  
brew install opam
```

2 Énoncés

Pour chaque mini-TP, il est demandé de bien tester son code sur des petits exemples, pour s'assurer qu'il a, au moins en première approche, le bon comportement. La réalisation autonome des tests fait partie de l'évaluation des TP aux concours. Attention, dans certains cas, créer les éléments nécessaires aux tests peut prendre plus de temps que d'écrire les fonctions demandées (mais ça fait partie du jeu).

N'essayez pas de tout faire d'un coup, c'est contreproductif. Essayez de vous tenir à un rythme d'un TP par semaine.

2.1 Parcours en profondeur

À réaliser en OCaml.

On implémente un graphe avec des listes d'adjacence :

```
type graphe = int list array
```

Écrire une fonction `dfs (s0: int) (g: graphe) : int list` qui renvoie la liste des sommets atteint depuis s_0 dans l'ordre d'un parcours en profondeur dans le graphe G .

2.2 Tas binaires

À réaliser en C.

On implémente un tas binaire min d'entiers dans un tableau avec le type suivant :

```
struct Tas {  
    int capa;  
    int size;  
    int* data;  
};  
  
typedef struct Tas tas;
```

tel que si t est de type `tas`, alors `t.capa` est la capacité, c'est-à-dire le nombre d'éléments maximal que peut contenir le tas, `t.size` est la taille, c'est-à-dire le nombre actuel d'éléments dans le tas, et `t.data` est le tableau des données correspondant aux valeurs du tas.

Écrire des fonctions `void inserer(tas* t, int x)` et `int extraire_min(tas* t)` qui font respectivement une insertion et l'extraction du minimum dans un tas. On ne gèrera pas les cas des tas vides ou pleins.

2.3 Satisfiabilité

À réaliser en OCaml.

On encode une formule propositionnelle sur un ensemble de variables $\mathcal{V} = \{x_0, \dots, x_{n-1}\}$ par :

```
type formule =  
  | Bot | Top  
  | Var of int  
  | Neg of formule  
  | Et of formule * formule  
  | Ou of formule * formule
```

tel que `Bot` et `Top` représentent $\perp$ et $\top$, `Var i` représente x_i et les autres connecteurs sont transparents.

Écrire, en procédant par retour sur trace (sans élagage), une fonction `sat (phi: formule) (n: int) : bool` qui teste si une formule φ sur l'ensemble de variables $\mathcal{V} = \{x_0, \dots, x_{n-1}\}$ est satisfiable. On supposera qu'aucune variable avec un numéro en dehors de $\llbracket 0, n-1 \rrbracket$ n'apparaît.

Indication : on pourra introduire un type de valuation et faire le retour sur trace sur les valuations.

2.4 Connexité

À réaliser en C.

On représente un graphe non orienté par matrice d'adjacence par :

```
typedef struct Graphe {
    int n;
    bool** mat;
} graphe;
```

tel que si G est un graphe représenté par `G`, alors `G.n` est son nombre de sommets, et `G.mat[s][t]` vaut `true` si et seulement s'il existe une arête de s à t .

Écrire une fonction `bool connexe(graphe G)` qui teste si un graphe est connexe.

Rappel : pour manipuler les booléens en tant que tels, il faut inclure `stdbool.h`.

2.5 Arbres binaires de recherche

À réaliser en OCaml.

On encode des arbres binaires de recherche (ABR) de manière persistante par le type :

```
type abr =
| Vide
| N of abr * int * abr
```

Écrire des fonctions `insérer (x: int) : abr` et `supprimer (x: int) (a: abr) : abr` qui font l'insertion et la suppression d'un élément x dans un ABR a . La suppression ne modifiera pas l'arbre si x n'y est pas présent. On ne cherchera pas à faire d'équilibrage d'arbre.

2.6 Parcours en largeur

À réaliser en C.

On encode un graphe par tableau de listes d'adjacence par :

```
struct Graphe {
    int n;
    int* deg;
    int** adj;
};
```

tel que si `G` est de type `struct Graphe` qui encode un graphe $G = (S, A)$, alors `G.n` est la valeur $|S|$, et pour $s \in S$, `G.deg[s]` est le degré sortant de s , et `G.adj[s]` est un tableau de taille `G.deg[s]` contenant les voisins de s , dans un ordre arbitraire.

Écrire une fonction `int* bfs(struct Graphe G, int s0)` qui calcule et renvoie un tableau `dist` de taille `G.n` tel que pour $s \in S$, `dist[s]` est la distance de s_0 à s , si s est accessible depuis s_0 , et `-1` sinon.

Indication : inutile de coder un `struct` de file compliqué, on peut se contenter d'utiliser un tableau de taille n avec un indice de début et un indice de fin pour délimiter le contenu courant de la file.

2.7 Acyclicité

À réaliser en OCaml.

On implémente, comme précédemment, un graphe **orienté** avec des listes d'adjacence :

```
type graphe = int list array
```

Écrire une fonction `acyclique (g: graphe) : bool` qui teste si un graphe donné est sans cycle.

2.8 Arbres binaires de recherche (bis)

À réaliser en C.

On implémente des ABR de manière impérative (c'est-à-dire modifiable) par le type :

```
struct ABR {  
    int x;  
    struct ABR* g;  
    struct ABR* d;  
};  
  
typedef struct ABR abr;
```

tel que si `A` est de type `abr`, alors `A.x` est sa racine, `A.g` un pointeur vers son enfant gauche et `A.d` un pointeur vers son enfant droit.

Écrire des fonctions `bool recherche(abr* A, int x)` et `void insertion(abr* A, int x)` qui effectuent la recherche et l'insertion dans un ABR. On supposera, pour l'insertion, que l'arbre `A` est non vide.